Telecom Customer Churn with a Multi-Layer Perceptron Model

U21ADP05-Exploratory Data Analysis and Visualization

Submitted by

MITHUNA R 24CS118

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous Institution)

ARASUR, COIMBATORE 641 407

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

Exploratory Data Analysis and Visualization of Telecom Customer Churn with a Multi-Layer Perceptron Model

Mithuna R(24cs118)
Department of Computer Science and Engineering
KPR Institute of Engineering and Technology
Coimbatore, India

GITHUB LINK :https://github.com/Mithuna713/churn-eda-visualization.

Abstract

Customer churn is a costly problem for subscription-based businesses, and understanding why customers leave is the first step toward retaining them. This paper presents an exploratory data analysis and visualization study of the IBM Telco Customer Churn dataset, which contains 7,043 customers described by 21 fields. The data were parsed, cleaned, encoded and divided into training, validation and test sets, and a series of visualizations was produced following the seven stages of data visualization. Statistical tests and probability distribution fitting show that churn is strongly associated with short tenure, month-to-month contracts, fiber optic service and electronic check payment, and that the number of add-on services is over-dispersed relative to a Binomial model. A multi-layer perceptron trained with dropout, class weights, early stopping and a 12-point hyperparameter grid search reaches a test AUC of 0.842 and a recall of 0.786, matching a logistic regression baseline. The results indicate that new customers on flexible contracts form the highest-risk segment and support targeted retention actions.

Index Terms—churn prediction, data visualization, exploratory data analysis, multi-layer perceptron, probability distributions.

I. INTRODUCTION

Customer churn, the loss of subscribers who cancel a service, is a major cause of revenue loss in telecommunication and other subscription-based industries, because acquiring a new customer is generally more costly than retaining an existing one. A company that can recognize which customers are likely to leave, and understand why, can offer a suitable plan, support or discount before the customer cancels.

Exploratory data analysis (EDA) uses numerical summaries and graphics to understand the structure of data, detect quality problems and form hypotheses before any model is built [2]. Fry describes the process of turning data into a visual as seven connected stages: acquire, parse, filter, mine, represent, refine and interact [3]. This paper applies these stages to a real-world customer dataset and then trains a multi-layer perceptron (MLP) [4] to predict churn. The analysis was implemented in Python using pandas, scikit-learn [5], TensorFlow/Keras [6], Matplotlib [7] and Seaborn [8].

The main contributions of this work are:

- a data-quality and preprocessing pipeline for the Telco dataset, including handling of blank TotalCharges values, redundant category levels, encoding, scaling and stratified splitting;
- a set of visualizations covering univariate, bivariate and multivariate analysis, with statistical tests and discrete and continuous distribution fitting;
- a tuned MLP classifier evaluated with loss and accuracy curves, a confusion matrix, ROC analysis and permutation feature importance, compared with a logistic regression baseline; and
- practical retention recommendations derived from the findings.

II.OBJECTIVES

The main aim of this case study is to apply exploratory data analysis and visualization to a real-world customer dataset, to understand why customers leave a telecom service, and to build a deep learning model that predicts customer churn.

The specific objectives are:

- To select a real-world dataset with at least 15 fields and understand its structure, quality and statistics.
- To handle missing values, duplicates, outliers, categorical encoding and scaling, and to split the data into training, validation and test sets.
- To create at least five meaningful visualizations covering the seven stages of data visualization, and to explain what each one shows, why it was created and what insight it gives.
- To carry out univariate, bivariate and multivariate analysis, and to study discrete and continuous probability distributions in the data.
- To build a deep learning model (Multi-Layer Perceptron) with feature analysis and basic hyperparameter tuning, and to evaluate it with

loss, accuracy, confusion matrix, ROC and AUC visualizations.
- To draw practical conclusions and recommendations for customer retention.

III. DATASET PREPROCESSING

A. Dataset

The Telco Customer Churn dataset was released by IBM as a public sample dataset [1]. Each of the 7,043 rows represents one customer of a telecom provider offering phone and internet services, and the 21 fields are grouped in Table I. The target field, Churn, records whether the customer left within the last month. Of all customers, 1,869 (26.5%) churned and 5,174 (73.5%) stayed, so the target is moderately imbalanced.

TABLE I
FIELDS OF THE DATASET

Group	Fields
Identifier	customerID (dropped before modeling)
Demographics	gender, SeniorCitizen, Partner, Dependents
Account	tenure, Contract, PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges
Services	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies
Target	Churn (Yes/No)

IV.PREPROCESSING

A. Data Cleaning

TotalCharges was stored as text because 11 rows held a blank string. After conversion to a numeric type, all 11 missing values were found to belong to customers with tenure equal to zero, that is, customers who joined in the current month and had not yet been billed. These values were therefore filled with 0 rather than with the mean or median.

No fully duplicated rows were found and every customerID is unique. Although 22 rows share identical feature values when the identifier is ignored, they were retained because they correspond to different customers with the same plan and profile. The interquartile-range (IQR) rule with a factor of 1.5 flagged 0 outliers in tenure, 0 in MonthlyCharges and 0 in TotalCharges, so no rows were removed.

The levels “No phone service” and “No internet service” were merged into “No”, since this information is already held by PhoneService and InternetService. A new feature, n_addons, counting the add-on services subscribed (0 to 6), was engineered from the six add-on fields.

C. Encoding, Scaling and Splitting

Binary fields were mapped to 0/1, and InternetService, Contract and PaymentMethod were one-hot encoded with the first level dropped, giving 24 input features. The fields tenure, MonthlyCharges, TotalCharges and n_addons were standardized to zero mean and unit variance, with the scaler fitted on the training data only to prevent leakage. A stratified 70:15:15 split produced 4,930 training, 1,056 validation and 1,057 test records, each with a churn rate of about 26.5%.

V. VISUALIZATION METHODOLOGY

Charts were produced with Matplotlib and Seaborn, and one interactive chart was produced with Plotly. A consistent color scheme is used throughout, with blue for retained customers and red for churned customers. Table II summarizes how each stage of the seven-stage process was carried out.

TABLE II
SEVEN STAGES OF VISUALIZATION APPLIED

Stage	Implementation
Acquire	Downloaded the public Telco CSV file from IBM's repository.
Parse	Read the file with pandas and corrected field types.
Filter	Handled blanks, duplicates and outliers; dropped the identifier; merged redundant levels.
Mine	Computed descriptive statistics, skewness, chi-square and Mann–Whitney tests, correlations and distribution fits.
Represent	Chose histograms, box plots, bar charts, heatmap, scatter and pair plots, and probability plots.
Refine	Added titles, labels, legends, reference lines and a consistent color scheme.
Interact	Built an interactive Plotly scatter chart with hover, zoom and legend filtering.

VI. EXPLORATORY ANALYSIS

A. Class Balance and Univariate Distributions

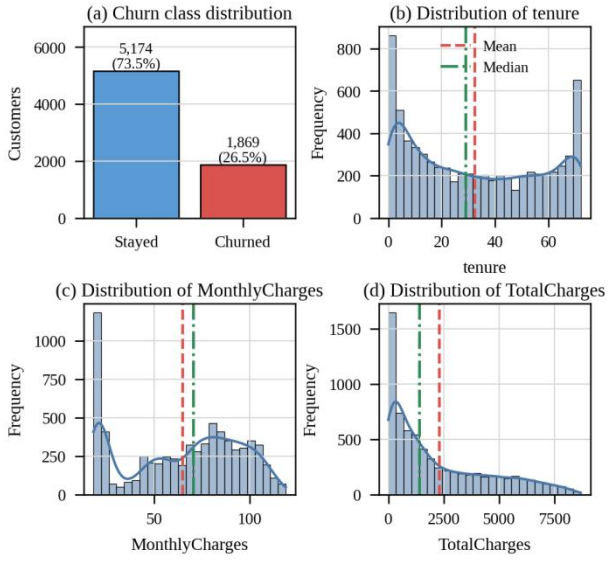


Fig. 1. (a) Class distribution of the target and (b)–(d) histograms of the numerical fields with mean (dashed) and median (dash-dot) lines.

Fig. 1(a) shows that about 26.5% of customers churned, a ratio of roughly 2.8:1. A model that always predicts “stay” would be 73.5% accurate yet useless for retention, so recall, F1-score and AUC are reported together with accuracy, and class weights are used in training.

Tenure is U-shaped, with one group of new customers and one at the 72-month maximum (mean 32.4, median 29.0; skewness 0.24). MonthlyCharges is bimodal, with a cluster near 20 for basic plans and a broad hump between 70 and 100 (skewness -0.22). TotalCharges is right-skewed

(skewness 0.96), with the mean (2,279.7) well above the median (1,394.6). None of the three is normally distributed, which motivates standardization and non-parametric tests.

B. Numerical Fields by Churn Status

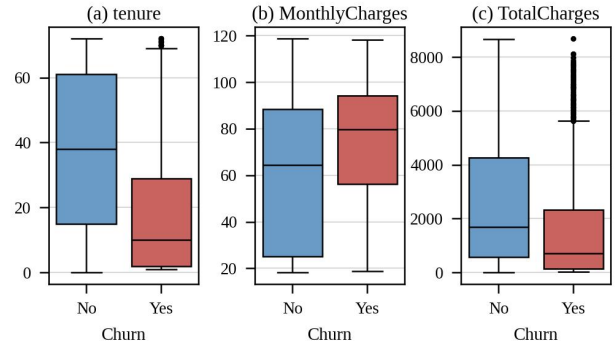


Fig. 2. Box plots of (a) tenure, (b) MonthlyCharges and (c) TotalCharges grouped by churn status.

Churners have a much lower median tenure (10 versus 38 months), a higher median monthly charge (79.65 versus 64.43) and a lower median total charge (703.55 versus 1,679.52), as shown in Fig. 2. The few points beyond the whiskers within the churned group are genuine long-tenure or high-billing customers, and the IQR check on the full dataset found no outliers.

C. Churn Rate by Customer Segment

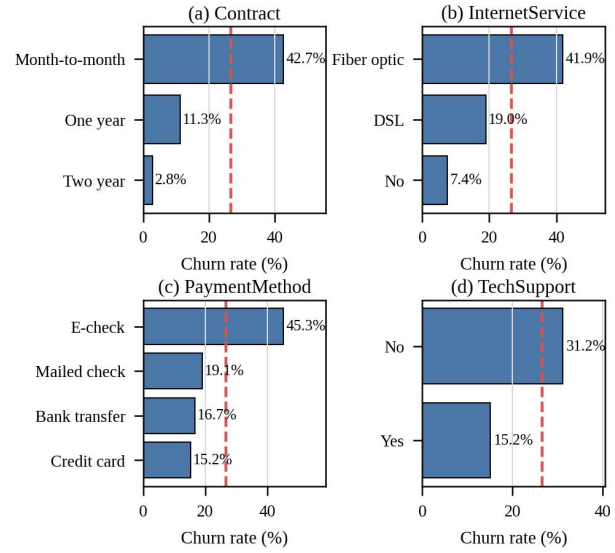


Fig. 3. Churn rate by (a) contract type, (b) internet service, (c) payment method and (d) tech support. The dashed line marks the overall churn rate.

Month-to-month customers churn at 42.7%, compared with 11.3% on one-year and 2.8% on two-year contracts (Fig. 3). Fiber optic users churn at 41.9% against 19.0% for DSL. Electronic check payers churn at 45.3%, about three times the rate of automatic payment users, and customers without tech support churn at 31.2% against 15.2% with it. Churn also falls steadily with tenure: 47.4% in the first 12 months, 28.7% in months 13 to 24, 20.4% in months 25 to 48 and 9.5% beyond 48 months.

D. Correlation Analysis

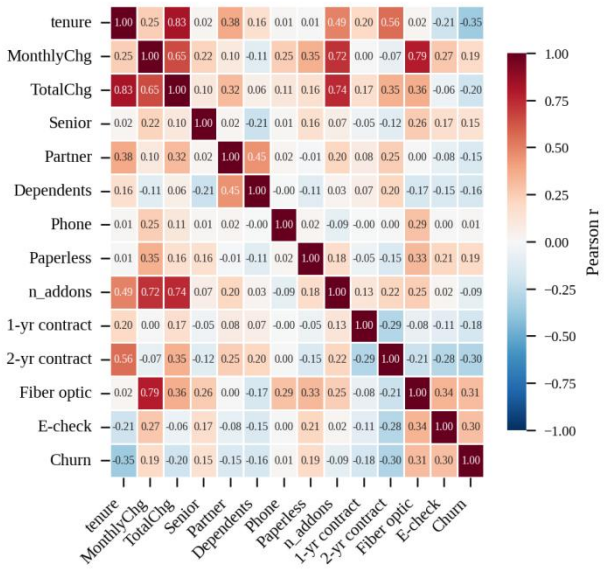


Fig. 4. Pearson correlation heatmap of key features and churn.

Fig. 4 shows that the strongest correlations with churn are tenure (−0.35), fiber optic service (+0.31), electronic check payment (+0.30) and a two-year contract (−0.30), while PhoneService is almost uncorrelated (+0.01). Several predictors overlap: tenure with TotalCharges (0.83), MonthlyCharges with fiber optic service (0.79) and MonthlyCharges with n_addons (0.72). This multicollinearity is tolerable for a neural network but would require care in a purely linear model.

E. Bivariate and Multivariate Relationships

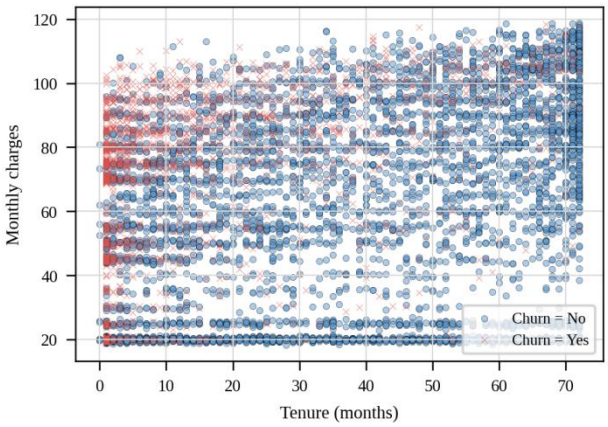


Fig. 5. Monthly charges against tenure for customers who stayed (blue circles) and churned (red crosses).

In Fig. 5, churners concentrate in the upper-left region, where tenure is low (mostly under 20 months) and monthly charges are high (above about 65). Long-tenure customers rarely churn at any price level, and a band near a monthly charge of 20 consists of long-standing phone-only customers who almost never leave.

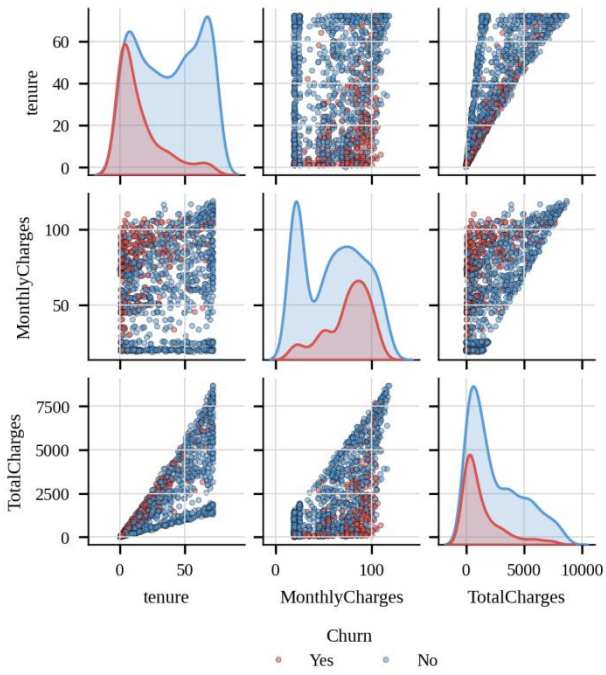


Fig. 6. Pair plot of tenure, MonthlyCharges and TotalCharges for a random sample of 1,200 customers.

The pair plot in Fig. 6 confirms that churners peak at very low tenure and at high monthly charges. TotalCharges against tenure forms a funnel because total charges are roughly tenure multiplied by the monthly charge, and churned points sit at its narrow, low end. No single pair separates the classes perfectly, so a multivariate model is needed.

F. Discrete Distribution

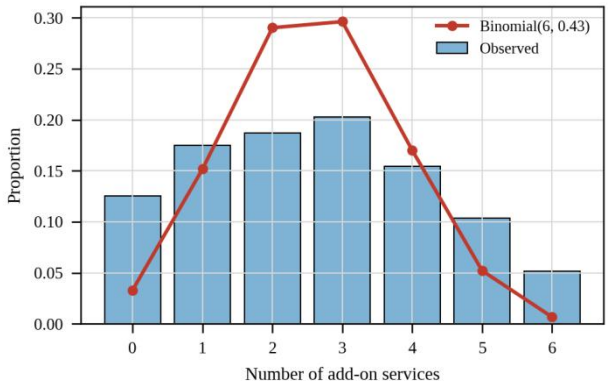


Fig. 7. Number of add-on services among internet customers compared with a fitted Binomial distribution.

For the 5,517 customers with internet service, the number of add-on services was compared with a Binomial(6, 0.434) distribution (Fig. 7). The fit is poor (chi-square goodness-of-fit statistic 3,774). The observed variance (2.892) is almost twice the Binomial variance (1.474), which indicates over-dispersion: too many customers take no add-ons (12.6% observed against 3.3% expected) or all six (5.1% against 0.7%). Add-on choices are therefore not independent, which suggests distinct customer segments.

G. Continuous Distributions

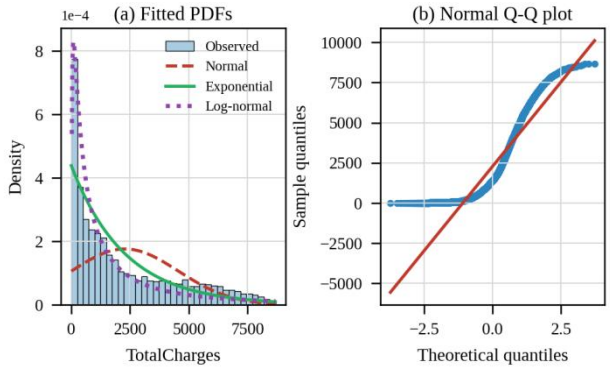


Fig. 8. (a) TotalCharges with fitted Normal, Exponential and Log-normal densities and (b) normal Q–Q plot.

The Kolmogorov–Smirnov distance for TotalCharges is 0.158 for the Normal fit, 0.093 for the Exponential and 0.086 for the Log-normal (Fig. 8), so the Normal fit is clearly the worst. The mean (2279.7) is almost equal to the standard deviation (2266.6), which is typical of an exponential-like, right-skewed variable, and the Q–Q plot departs from the line in the upper tail. MonthlyCharges is also non-normal (Shapiro–Wilk W = 0.921 on a random sample of 2,000) because of its two-peak shape.

H. Statistical Tests

A chi-square test of independence with Churn was run for each categorical field, with association strength measured by Cramér's V (Table III). All differences are significant at the 0.1% level, and contract type has the strongest association.

TABLE III
CHI-SQUARE TESTS AGAINST CHURN

Field	χ^2	d.f.	Cramér's V
Contract	1,184.6	2	0.410
InternetService	732.3	2	0.322
PaymentMethod	648.1	3	0.303
PaperlessBilling	258.3	1	0.191
OnlineSecurity	205.6	1	0.171
Dependents	189.1	1	0.164

For the numerical fields, the Mann–Whitney U test was used because the distributions are not normal. Tenure, MonthlyCharges and TotalCharges all differ significantly between churners and non-churners ($p < 0.001$), with point-biserial correlations of -0.352, 0.193 and -0.198 respectively.

VII. DEEP LEARNING MODEL

A. Architecture and Training

An MLP was selected because the data are tabular with numerical and encoded categorical inputs. The final network (Table IV) has two hidden layers of 32 and 16 ReLU units, each followed by dropout [10], and a single sigmoid output giving the probability of churn. It has 1,345 trainable parameters. Training used binary cross-entropy loss and the Adam optimizer [9] with early stopping on validation loss. Class weights of 0.681 (stayed) and 1.885 (churned) were applied to compensate for the imbalance.

TABLE IV
FINAL MODEL CONFIGURATION

Parameter	Value
Input features	24
Hidden layers	32 – 16 (ReLU)
Dropout rate	0.2
Optimizer / learning rate	Adam / 0.001
Batch size / max epochs	64 / 150
Early stopping	Patience 10, best weights restored
Epochs run / best epoch	33 / 23
Random seed	42

B. Hyperparameter Tuning

A grid search over three hidden-layer sizes, two dropout rates and two learning rates (12 combinations, Table V) was run, and the configuration with the highest validation AUC (0.8452) was selected. The test set was not used during tuning. Differences between configurations are small, and the larger 128–64 networks were slightly worse, which suggests that this dataset does not require a large network.

TABLE V
GRID SEARCH RESULTS

Units	Dropout	LR	Epochs	Val. AUC
32–16	0.2	0.001	33	**0.8452**
32–16	0.2	0.005	14	0.8441
32–16	0.4	0.001	71	0.8442
32–16	0.4	0.005	15	0.8411
64–32	0.2	0.001	30	0.8445
64–32	0.2	0.005	24	0.8405
64–32	0.4	0.001	48	0.8438
64–32	0.4	0.005	27	0.8404
128–64	0.2	0.001	38	0.8358
128–64	0.2	0.005	20	0.8371
128–64	0.4	0.001	30	0.8428
128–64	0.4	0.005	22	0.8373

C. Feature Analysis

Features were analyzed in two ways: through the correlations and tests of Section IV, and through permutation importance on the test set, in which each feature is shuffled in turn and the resulting fall in AUC is recorded.

VIII. RESULTS AND DISCUSSION

Test-set performance of the MLP and of a logistic regression baseline is given in Table VI.

TABLE VI
TEST-SET PERFORMANCE

Metric	MLP	Logistic reg.
Accuracy	0.747	0.747
Precision (churn)	0.515	–
Recall (churn)	0.786	0.779
F1-score (churn)	0.623	–
ROC AUC	0.842	0.845
Average precision	0.650	–

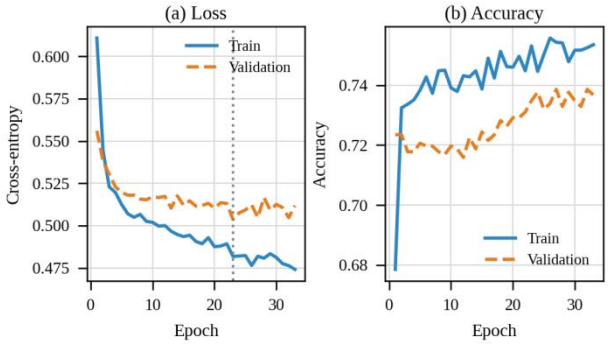


Fig. 9. Training and validation (a) loss and (b) accuracy against epoch. The dotted line marks the best epoch.

Both losses fall quickly at first (Fig. 9). After about epoch 23 the training loss keeps decreasing while the validation loss stops improving, which marks the onset of overfitting. Early stopping ended training at epoch 33 and restored the weights of epoch 23. Accuracy settles at about 75.4% for training and 73.7% for validation, and the small gap shows no serious overfitting. It is lower than an accuracy-only model would give because class weights deliberately trade some accuracy for better detection of churners.

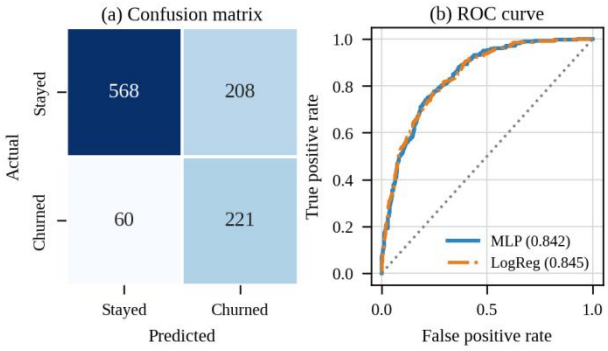


Fig. 10. Test-set (a) confusion matrix of the MLP at threshold 0.5 and (b) ROC curves of the MLP and the logistic regression baseline.

The model correctly identified 221 of the 281 customers who churned (recall 78.6%) and missed 60. It kept 568 of 776 loyal customers, but wrongly flagged 208 of them, giving a precision of 51.5% (Fig. 10(a)). For retention this is a sensible trade-off, because contacting a loyal customer costs far less than losing a customer who was never contacted. The MLP reaches an AUC of 0.842 (Fig. 10(b)): a randomly chosen churner receives a higher churn probability than a randomly chosen loyal customer about 84.2% of the time. The logistic regression baseline has almost the same curve (AUC 0.845), so the neural network gives no measurable gain over a linear model on this dataset. The main churn signals therefore act mostly in a simple additive way.

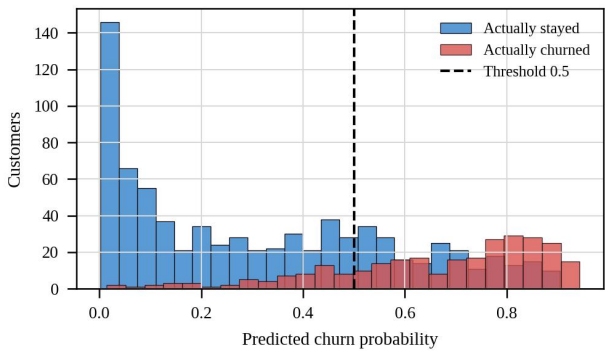


Fig. 11. Distribution of predicted churn probabilities for customers who actually stayed and churned.

Customers who stayed concentrate at low predicted probabilities and churners spread toward high probabilities (Fig. 11). The overlap near the middle explains the false positives and false negatives. Raising the threshold would increase precision and lower recall, so it can be set according to the cost of a retention offer.

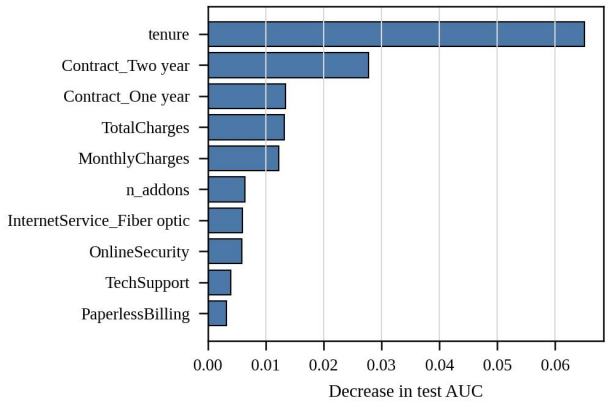


Fig. 12. Top 10 features by permutation importance on the test set.

Tenure is by far the most important feature (AUC drop 0.065), followed by the two-year contract (0.028), the one-year contract (0.013), TotalCharges (0.013) and MonthlyCharges (0.012), as shown in Fig. 12. This ranking agrees with the chi-square tests and the correlation heatmap, so the model relies on the same business signals found by the exploratory analysis. Because correlated features such as tenure and TotalCharges can share information, importance values should be read as approximate.

IX. CONCLUSION AND FUTURE SCOPE

This paper presented an exploratory analysis and visualization study of the Telco Customer Churn dataset, followed by an MLP classifier. The analysis showed that new customers are the most vulnerable (churn of 47.4% in the first year against 9.5% after four years), that month-to-month contracts, fiber optic service and electronic check payment mark high-risk segments, and that tech support and online security are associated with lower churn. The tuned MLP achieved an accuracy of 74.7%, a recall of 0.786, an F1-score of 0.623 and an AUC of 0.842 on unseen data, practically equal to a logistic regression baseline.

The findings suggest an onboarding and engagement programme for customers in their first year, incentives to move from month-to-month to longer contracts and from electronic check to automatic payment, bundling of tech support and online security with fiber optic plans, and the use of predicted churn probability to prioritize outreach.

The study has limitations. The data are a single snapshot, so the results show association rather than cause; usage, complaint and competitor information is absent; and the data are a public sample that should be validated on a company's own records. Future work includes comparison with gradient-boosted trees, resampling or focal loss for imbalance, k-fold cross-validation, SHAP explanations, richer behavioral features, survival analysis of time to churn and an interactive dashboard for retention teams.

REFERENCES

[1] IBM, “Telco customer churn (sample dataset),” GitHub repository IBM/telco-customer-churn-on-icp4d. [Online]. Available: https://github.com/IBM/telco-customer-churn-on-icp4d

[2] J. W. Tukey, Exploratory Data Analysis. Reading, MA, USA: Addison-Wesley, 1977.

[3] B. Fry, Visualizing Data: Exploring and Explaining Data with the Processing Environment. Sebastopol, CA, USA: O'Reilly Media, 2007.

[4] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[5] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[6] M. Abadi et al., “TensorFlow: A system for large-scale machine learning,” in Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI), Savannah, GA, USA, 2016, pp. 265–283.

[7] J. D. Hunter, “Matplotlib: A 2D graphics environment,” Comput. Sci. Eng., vol. 9, no. 3, pp. 90–95, 2007.

[8] M. L. Waskom, “seaborn: Statistical data visualization,” J. Open Source Softw., vol. 6, no. 60, p. 3021, 2021.

[9] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. 3rd Int. Conf. Learning Representations (ICLR), San Diego, CA, USA, 2015.

[10] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” J. Mach. Learn. Res., vol. 15, pp. 1929–1958, 2014.

Appendix (Code Section)

Case Study: Exploratory Data Analysis and Visualization of a Real-World Dataset

Course: U21ADP05-Exploratory Data Analysis and Visualization

Dataset : IBM Telco Customer Churn (7,043 customers x 21 fields)

Task : Predict whether a telecom customer will churn (binary classification)

Model : Multi-Layer Perceptron (Keras / TensorFlow)

Pipeline follows the 7 stages of data visualization (Ben Fry):
1 Acquire -> 2 Parse -> 3 Filter -> 4 Mine -> 5 Represent -> 6 Refine -> 7 Interact

```
import os, json, random, warnings, itertools
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score,precision_score, recall_score,
    f1_score,roc_auc_score,roc_curve,confusion_matrix,
    average_precision_score)

import tensorflow as tf
from tensorflow import keras
warnings.filterwarnings("ignore")
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"

SEED = 42
random.seed(SEED);
np.random.seed(SEED);
keras.utils.set_random_seed(SEED)
FIG_DIR = "figs";
os.makedirs(FIG_DIR, exist_ok=True)
RESULTS = {}
```

[Plot style: Times-style serif font, black text, colour palet]t

```
plt.rcParams.update({
   "font.family": "serif",
   "font.serif": ["Times New Roman", "Liberation Serif", "DejaVu Serif"],
   "text.color":"black","axes.labelcolor":"black" "axes.edgecolor": "black",
   "xtick.color": "black", "ytick.color": "black", "axes.titlecolor": "black",
   "axes.titlesize": 12, "axes.labelsize": 11, "legend.fontsize": 9,
   "axes.grid": True, "grid.color": "0.85", "grid.linewidth": 0.5,
   "figure.dpi": 120, "savefig.dpi": 200, "savefig.bbox": "tight",
})

C_NO, C_YES = "#5B9BD5", "#D9534F" # blue = stayed, red = churned
PAL = {"No": C_NO, "Yes": C_YES}

def save(name):
 plt.tight_layout()
 plt.savefig(os.path.join(FIG_DIR, name)); plt.close()
```

STAGE 1 - ACQUIRE : load the raw dataset

```
URL = ("https://raw.githubusercontent.com/IBM/telco-customer-churn-
    on-icp4d/"
    "master/data/Telco-Customer-Churn.csv")
df = pd.read_csv("telco.csv")
if os.path.exists("telco.csv")
 else pd.read_csv(URL)
RESULTS["shape_raw"] = list(df.shape)
RESULTS["dtypes_raw"] = {c: str(t) for c, t in df.dtypes.items()}
```

STAGE 2 - PARSE : fix data types

```
# TotalCharges is stored as text because 11 rows contain a blank string

blank_total = (df["TotalCharges"].astype(str).str.strip() == "").sum()
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df["SeniorCitizen"] = df["SeniorCitizen"].map({0: "No", 1: "Yes"})
RESULTS["blank_totalcharges"] = int(blank_total)
```

STAGE 3 - FILTER : missing values, duplicates, outliers, cleaning

```
RESULTS["missing_after_parse"] ={c:int(v)
for c, v in df.isna().sum().items()
 if v > 0}

# The blank rows are customers with tenure = 0 (joined this month,
    nothing billed yet)

RESULTS["blank_rows_tenure"]=df.loc[df["TotalCharges"].isna(),
"tenure"].tolist()
df["TotalCharges"] = df["TotalCharges"].fillna(0.0)
RESULTS["duplicate_rows_full"] = int(df.duplicated().sum())
RESULTS["customerid_unique"]  = bool(df["customerID"].is_unique)
RESULTS["duplicate_rows_excluding_id"]=int(df.drop(columns="cusom
    erID").duplicated().sum())
num_cols = ["tenure", "MonthlyCharges", "TotalCharges"]
outliers = {}
for c in num_cols:
 q1, q3 = df[c].quantile([.25, .75]); iqr = q3 - q1
  lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
 outliers[c] = {"lower": float(lo), "upper": float(hi),
"count": int(((df[c] < lo) | (df[c] > hi)).sum())}
RESULTS["outliers_iqr"] = outliers
```

Cleaning: collapse redundant levels (information already held by other columns)

```
for c in ["MultipleLines"]:
df[c] = df[c].replace("No phone service", "No")
addon_cols = ["OnlineSecurity", "OnlineBackup", "DeviceProtection",
 "TechSupport", "StreamingTV", "StreamingMovies"]
for c in addon_cols:
df[c] = df[c].replace("No internet service", "No")
df["Churn_bin"] = (df["Churn"] == "Yes").astype(int)
```

STAGE 4 - MINE : descriptive statistics and hypothesis tests

```
RESULTS["describe"] = df[num_cols].describe().round(2).to_dict()
RESULTS["skew"] = {c: round(float(df[c].skew()), 3) for c in num_cols}
RESULTS["kurtosis"] = {c: round(float(df[c].kurt()), 3) for c in num_cols}
RESULTS["churn_rate"] = round(float(df["Churn_bin"].mean()), 4)
RESULTS["churn_counts"] = df["Churn"].value_counts().to_dict()
RESULTS["group_means"]
    =df.groupby("Churn")[num_cols].mean().round(2).to_dict()
```

```
RESULTS["group_medians"]
    =df.groupby("Churn")[num_cols].median().round(2).to_dict()

cat_cols=[c for c in df.columns if not pd.api.types.is_numeric_dtype(df[c])
    and c not in ["customerID", "Churn", "tenure_band"]]



chi_rows = []
for c in cat_cols:
 ct = pd.crosstab(df[c], df["Churn"])
 chi2, p, dof, _ = stats.chi2_contingency(ct)
 v = np.sqrt(chi2 / (ct.values.sum() * (min(ct.shape) - 1)))
 chi_rows.append({"feature":c,"chi2":round(float(chi2), 1), "dof": int(dof),
"p": float(p), "cramers_v": round(float(v), 3)})
chi_rows.sort(key=lambda r: -r["cramers_v"])
RESULTS["chi_square"] = chi_rows

mw = {}
for c in num_cols:
 a = df.loc[df.Churn == "Yes", c]; b = df.loc[df.Churn == "No", c]
 u, p = stats.mannwhitneyu(a, b)
 mw[c] = {"U": float(u), "p": float(p)}
RESULTS["mann_whitney"] = mw
RESULTS["pointbiserial"]={c: round(float(df[c].corr(df["Churn_bin"])),3)
    for c in num_cols}

churn_by = {}
for c in ["Contract", "InternetService", "PaymentMethod", "TechSupport",
    "SeniorCitizen","PaperlessBilling","Partner","Dependents","Online
    Security"]:
churn_by[c]=(df.groupby(c)["Churn_bin"].mean()100).round(1).to_dict()
RESULTS["churn_rate_by"] = churn_by

# tenure bands
df["tenure_band"]= pd.cut(df["tenure"], [-1, 12, 24, 48, 72],
            labels=["012", "13-24", "25-48", "49-72"])
RESULTS["churn_by_tenure_band"]=(df.groupby("tenure_band")["Chur
    n_bin"].mean() * 100).round(1).to_dict()
```

STAGE 5 & 6 - REPRESENT + REFINE : visualisations (colour, labelled)

Fig 1: class balance

```
fig, ax = plt.subplots(figsize=(4.6, 3.6))
vc = df["Churn"].value_counts().reindex(["No", "Yes"])
bars = ax.bar(["Stayed (No)", "Churned (Yes)"], vc.values,
 color=[C_NO, C_YES], edgecolor="black")
for b, v in zip(bars, vc.values):
ax.text(b.get_x() + b.get_width() / 2, v + 60, f"{v:,}\n
({v / len(df) * 100:.1f}%)", ha="center", fontsize=10)
ax.set_ylabel("Number of customers");
ax.set_title("Target Class Distribution (Churn)")
ax.set_ylim(0, vc.max() * 1.18);
ax.grid(axis="x", visible=False)
save("fig01_class_distribution.png")
```

Fig 2: univariate histograms

```
fig, axs = plt.subplots(1, 3, figsize=(11, 3.4))
for ax,c,u in zip(axs, num_cols,
["months","INR/USD per month","total billed"]):
sns.histplot(df[c],bins=30,kde=True,
color="#4C78A8",
edgecolor="black",
ax=ax, line_kws={"color": "#1F3A5F", "lw": 1.8})
ax.axvline(df[c].mean(), color="#D9534F", ls="--", lw=1.6,
 label=f"Mean = {df[c].mean():.1f}")
 ax.axvline(df[c].median(),color="#2E8B57",ls="-.",lw=1.6,
     label=f"Median = {df[c].median():.1f}")
 ax.set_title(f"Distribution of {c}");
 ax.set_xlabel(c);
ax.set_ylabel("Frequency"); ax.legend()
save("fig02_numeric_histograms.png")
```

Fig 3: boxplots by churn fig,

```
 axs = plt.subplots(1, 3, figsize=(11, 3.6))
for ax, c in zip(axs, num_cols):
sns.boxplot(data=df, x="Churn", y=c, order=["No", "Yes"],
 palette=PAL, linecolor="black",
 linewidth=1.1, ax=ax,
 flierprops={"marker": "o","markersize": 3, "markerfacecolor": "black"})
 ax.set_title(f"{c}byChurnStatus");
ax.set_xlabel("Churn"); ax.set_ylabel(c)
save("fig03_boxplots_by_churn.png")
```

Fig 4: churn rate by categorical features

```
fig, axs = plt.subplots(2, 2, figsize=(11, 7))
for ax, c in zip(axs.ravel(), ["Contract", "InternetService",
    "PaymentMethod", "TechSupport"]):
 rate=(df.groupby(c)["Churn_bin"].mean()                    *
    100).sort_values(ascending=False)
  ax.barh(rate.index, rate.values, color="#4C78A8", edgecolor="black")
  for i, v in enumerate(rate.values):
  ax.text(v + 0.8, i, f"{v:.1f}%", va="center", fontsize=9)
  ax.axvline(df["Churn_bin"].mean() * 100, color="#D9534F", ls="--",
    lw=1.5)
 ax.set_xlim(0, rate.max() * 1.18);
 ax.invert_yaxis()
 ax.set_title(f"Churn Rate by {c}");
ax.set_xlabel("Churn rate (%)");
ax.grid(axis="y", visible=False)
axs[0, 0].text(df["Churn_bin"].mean() * 100 + 0.5, -0.45, "dashed line =
    overall churn rate", fontsize=8)
save("fig04_churn_rate_by_category.png")
```

Encoding (needed for correlation heatmap and model)

```
X=df.drop(columns=["customerID","Churn","Churn_bin","tenure_band"])
    .copy()
binary_map = {"Yes": 1, "No": 0, "Male": 1, "Female": 0}
for c in X.columns:
  if not pd.api.types.is_numeric_dtype(X[c]) and set(X[c].unique()) <=
    set(binary_map):
 X[c] = X[c].map(binary_map)
X["n_addons"] = (df[addon_cols] == "Yes").sum(axis=1)
engineered feature
multi = [c for c in X.columns if not pd.api.types.is_numeric_dtype(X[c])]
X = pd.get_dummies(X, columns=multi, drop_first=True, dtype=int)
y = df["Churn_bin"].values
RESULTS["n_features_encoded"] = int(X.shape[1])
RESULTS["feature_names"] = list(X.columns)
```

Fig 5: correlation heatmap

```
sel = ["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen",
    "Partner", "Dependents",
   "PhoneService", "PaperlessBilling", "n_addons", "Contract_One
    year", "Contract_Two year",
   "InternetService_Fiber optic", "PaymentMethod_Electronic check"]
cm = X[sel].assign(Churn=y).corr()
fig, ax = plt.subplots(figsize=(10, 8))
```

```
sns.heatmap(cm, annot=True, fmt=".2f", cmap="RdBu_r", vmin=-1, vmax=1, linewidths=.5, linecolor="white",
            annot_kws={"size": 8}, cbar_kws={"label": "Pearson correlation"}, ax=ax)
ax.set_title("Correlation Heatmap of Key Features and Churn"); ax.grid(False)
plt.xticks(rotation=45, ha="right")
save("fig05_correlation_heatmap.png")
RESULTS["corr_with_churn"] = cm["Churn"].drop("Churn").round(3).sort_values().to_dict()
```

Fig 6: scatter

```
fig, ax = plt.subplots(figsize=(7, 4.8))
for lab, m, col, a in [("No", "o", C_NO, .55), ("Yes", "x", C_YES, .75)]:
    s = df[df.Churn == lab]
    ax.scatter(s["tenure"], s["MonthlyCharges"], marker=m, c=col, alpha=a, s=18,
               edgecolors="black" if m == "o" else None, linewidths=.3,
               label=f"Churn = {lab}")
ax.set_xlabel("Tenure (months)"); ax.set_ylabel("Monthly charges")
ax.set_title("Monthly Charges vs Tenure, by Churn Status"); ax.legend()
save("fig06_scatter_tenure_monthly.png")
```

Fig 7: discrete distribution (add-on services ~ Binomial?)

```
net = df[df["InternetService"] != "No"]
k = (net[addon_cols] == "Yes").sum(axis=1)
n = len(addon_cols); p_hat = k.mean() / n
obs = k.value_counts(normalize=True).reindex(range(n + 1), fill_value=0)
exp = stats.binom.pmf(range(n + 1), n, p_hat)
fig, ax = plt.subplots(figsize=(6.4, 4))
ax.bar(range(n + 1), obs.values, color="#7FB3D5", edgecolor="black", label="Observed")
ax.plot(range(n + 1), exp, "o-", color="#C0392B", lw=2, label=f"Binomial(n=6, p={p_hat:.2f})")
ax.set_xlabel("Number of add-on services subscribed"); ax.set_ylabel("Proportion of internet customers")
ax.set_title("Add-on Service Count vs Fitted Binomial Distribution"); ax.legend()
save("fig07_discrete_binomial.png")
chi_stat = float((len(k) * (obs.values - exp) ** 2 / exp).sum())
RESULTS["binomial_fit"] = {"n_internet_customers": int(len(k)), "p_hat": round(float(p_hat), 3),
    "mean": round(float(k.mean()), 3), "var_observed": round(float(k.var()), 3),
    "var_binomial": round(float(n * p_hat * (1 - p_hat)), 3), "chi2_gof": round(chi_stat, 1),
    "observed": obs.round(3).to_dict(), "expected": dict(zip(range(n + 1), np.round(exp, 3)))}
```

Fig 8: continuous distribution fits + Q-Q

```
tc = df["TotalCharges"]; tcp = tc[tc > 0]
mu, sd = stats.norm.fit(tc); sh, loc, sc = stats.lognorm.fit(tcp, floc=0); lam = stats.expon.fit(tc, floc=0)[1]
fig, axs = plt.subplots(1, 2, figsize=(11, 4))
xs = np.linspace(0, tc.max(), 400)
axs[0].hist(tc, bins=40, density=True, color="#A9CCE3", edgecolor="black", label="Observed")
axs[0].plot(xs, stats.norm.pdf(xs, mu, sd), "--", color="#C0392B", lw=2, label="Normal fit")
axs[0].plot(xs, stats.expon.pdf(xs, 0, lam), "-", color="#27AE60", lw=2, label="Exponential fit")
axs[0].plot(xs[1:], stats.lognorm.pdf(xs[1:], sh, loc, sc), ":", color="#8E44AD", lw=2.4, label="Log-normal fit")
axs[0].set_xlabel("TotalCharges"); axs[0].set_ylabel("Density")
axs[0].set_title("TotalCharges with Fitted Continuous Distributions"); axs[0].legend()
(osm, osr), (slope, icpt, r) = stats.probplot(tc, dist="norm")
axs[1].scatter(osm, osr, s=8, c="#2E86C1"); axs[1].plot(osm, slope * np.array(osm) + icpt, "-", color="#C0392B", lw=1.8)
axs[1].set_xlabel("Theoretical quantiles (Normal)"); axs[1].set_ylabel("Sample quantiles")
axs[1].set_title("Q-Q Plot of TotalCharges")
save("fig08_continuous_fits_qq.png")
RESULTS["continuous_fit"] = {
  "KS_normal": round(float(stats.kstest(tc, "norm", args=(mu, sd)).statistic), 3),
  "KS_exponential": round(float(stats.kstest(tc, "expon", args=(0, lam)).statistic), 3),
  "KS_lognormal": round(float(stats.kstest(tcp, "lognorm", args=(sh, loc, sc)).statistic), 3),
  "normal_mu_sd": [round(mu, 1), round(sd, 1)], "exp_mean": round(lam, 1),
  "qq_r": round(float(r), 3)}
mc = df["MonthlyCharges"]
RESULTS["monthly_shapiro_sample"] = round(float(stats.shapiro(mc.sample(2000, random_state=SEED)).statistic), 3)
```

Fig 9: pairplot

```
samp = df.sample(1200, random_state=SEED)
g = sns.pairplot(samp, vars=num_cols, hue="Churn", palette=PAL, diag_kind="kde",
                 plot_kws={"alpha": .6, "s": 14, "edgecolor": "black", "linewidth": .3},
                 corner=False, height=2.2)
g.fig.suptitle("Pairplot of Numerical Features (sample of 1,200 customers)", y=1.02, fontsize=12)
g.savefig(os.path.join(FIG_DIR, "fig09_pairplot.png")); plt.close("all")
```

STAGE 7 - INTERACT : interactive HTML chart (hover, zoom, legend filtering)

```
    import plotly.express as px
    fig_i = px.scatter(df, x="tenure", y="MonthlyCharges", color="Churn", symbol="Contract",
                       hover_data=["InternetService", "PaymentMethod", "TotalCharges"],
                       title="Interactive: Monthly Charges vs Tenure")
    fig_i.write_html("interactive_tenure_vs_charges.html")
    RESULTS["interactive_html"] = True
except Exception as e:
    RESULTS["interactive_html"] = False
```

PREPROCESSING : split 70/15/15, standardise numeric columns (train stats only)

```
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.30, stratify=y, random_state=SEED)
X_va, X_te, y_va, y_te = train_test_split(X_tmp, y_tmp, test_size=0.50, stratify=y_tmp, random_state=SEED)
scale_cols = ["tenure", "MonthlyCharges", "TotalCharges", "n_addons"]
sc_ = StandardScaler().fit(X_tr[scale_cols])
for part in (X_tr, X_va, X_te):
    part[scale_cols] = sc_.transform(part[scale_cols])
RESULTS["split"] = {"train": len(X_tr), "val": len(X_va), "test": len(X_te),
                    "train_churn": round(float(y_tr.mean()), 4), "val_churn": round(float(y_va.mean()), 4),
                    "test_churn": round(float(y_te.mean()), 4)}
```

```
Xtr, Xva, Xte = X_tr.values.astype("float32"), X_va.values.astype("float32"), X_te.values.astype("float32")
```

DEEP LEARNING MODEL : MLP with basic hyper-parameter grid search

```
cw_neg = len(y_tr) / (2 * (y_tr == 0).sum()); cw_pos = len(y_tr) / (2 * (y_tr == 1).sum())
class_weight = {0: float(cw_neg), 1: float(cw_pos)}
RESULTS["class_weight"] = {k_: round(v, 3) for k_, v in class_weight.items()}

def build_mlp(n_in, units, dropout, lr):
    m = keras.Sequential([
        keras.layers.Input(shape=(n_in,)),
        keras.layers.Dense(units[0], activation="relu"), keras.layers.Dropout(dropout),
        keras.layers.Dense(units[1], activation="relu"), keras.layers.Dropout(dropout),
        keras.layers.Dense(1, activation="sigmoid")])
    m.compile(optimizer=keras.optimizers.Adam(lr), loss="binary_crossentropy",
              metrics=["accuracy", keras.metrics.AUC(name="auc")])
    return m

grid = list(itertools.product([(32, 16), (64, 32), (128, 64)], [0.2, 0.4], [1e-3, 5e-3]))
best = {"auc": -1}; hp_rows = []
for units, dr, lr in grid:
    keras.utils.set_random_seed(SEED)
    m = build_mlp(Xtr.shape[1], units, dr, lr)
    es = keras.callbacks.EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True)
    h = m.fit(Xtr, y_tr, validation_data=(Xva, y_va), epochs=150, batch_size=64,
              class_weight=class_weight, callbacks=[es], verbose=0)
    va_auc = roc_auc_score(y_va, m.predict(Xva, verbose=0).ravel())
    hp_rows.append({"units": f"{units[0]}-{units[1]}", "dropout": dr, "lr": lr,
                    "epochs": len(h.history["loss"]), "val_auc": round(float(va_auc), 4)})
    if va_auc > best["auc"]:
        best = {"auc": va_auc, "model": m, "hist": h.history, "cfg": (units, dr, lr)}
RESULTS["hparam_grid"] = hp_rows
model, hist = best["model"], best["hist"]
RESULTS["best_cfg"] = {"units": list(best["cfg"][0]), "dropout": best["cfg"][1], "lr": best["cfg"][2],
                       "val_auc": round(float(best["auc"]), 4), "epochs_run": len(hist["loss"]),
                       "params": int(model.count_params())}
```

EVALUATION

```
p_te = model.predict(Xte, verbose=0).ravel(); yhat = (p_te >= 0.5).astype(int)
RESULTS["test_metrics"] = {
    "accuracy": round(accuracy_score(y_te, yhat), 4), "precision": round(precision_score(y_te, yhat), 4),
    "recall": round(recall_score(y_te, yhat), 4), "f1": round(f1_score(y_te, yhat), 4),
    "auc": round(roc_auc_score(y_te, p_te), 4), "avg_precision": round(average_precision_score(y_te, p_te), 4)}
cmx = confusion_matrix(y_te, yhat); RESULTS["confusion_matrix"] = cmx.tolist()
lr_model = LogisticRegression(max_iter=2000, class_weight="balanced").fit(Xtr, y_tr)
p_lr = lr_model.predict_proba(Xte)[:, 1]
RESULTS["baseline_logreg"] = {"auc": round(roc_auc_score(y_te, p_lr), 4),
    "accuracy": round(accuracy_score(y_te, (p_lr >= .5).astype(int)), 4),
    "recall": round(recall_score(y_te, (p_lr >= .5).astype(int)), 4)}
RESULTS["train_final"] = {"loss": round(hist["loss"][-1], 4), "val_loss": round(hist["val_loss"][-1], 4),
                          "acc": round(hist["accuracy"][-1], 4), "val_acc": round(hist["val_accuracy"][-1], 4)}
RESULTS["best_epoch"] = int(np.argmin(hist["val_loss"]) + 1)

ep = np.arange(1, len(hist["loss"]) + 1)
# Fig 10: loss
fig, ax = plt.subplots(figsize=(6.4, 4))
ax.plot(ep, hist["loss"], "-", color="#2E86C1", lw=2, label="Training loss")
ax.plot(ep, hist["val_loss"], "--", color="#E67E22", lw=2, label="Validation loss")
ax.axvline(RESULTS["best_epoch"], color="#7F8C8D", ls=":", lw=1.5, label=f"Best epoch = {RESULTS['best_epoch']}")
ax.set_xlabel("Epoch"); ax.set_ylabel("Binary cross-entropy loss"); ax.set_title("Loss vs Epoch"); ax.legend()
save("fig10_loss_vs_epoch.png")
# Fig 11: accuracy
fig, ax = plt.subplots(figsize=(6.4, 4))
ax.plot(ep, hist["accuracy"], "-", color="#2E86C1", lw=2, label="Training accuracy")
ax.plot(ep, hist["val_accuracy"], "--", color="#E67E22", lw=2, label="Validation accuracy")
ax.set_xlabel("Epoch"); ax.set_ylabel("Accuracy"); ax.set_title("Accuracy vs Epoch"); ax.legend()
save("fig11_accuracy_vs_epoch.png")
# Fig 12: confusion matrix
fig, ax = plt.subplots(figsize=(4.8, 4.2))
sns.heatmap(cmx, annot=True, fmt="d", cmap="Blues", cbar=False, linewidths=1, linecolor="white",
            annot_kws={"size": 14}, xticklabels=["Stayed", "Churned"], yticklabels=["Stayed", "Churned"], ax=ax)
ax.set_xlabel("Predicted label"); ax.set_ylabel("Actual label"); ax.set_title("Confusion Matrix (Test Set)"); ax.grid(False)
save("fig12_confusion_matrix.png")
# Fig 13: ROC
fpr, tpr, _ = roc_curve(y_te, p_te); fpr2, tpr2, _ = roc_curve(y_te, p_lr)
fig, ax = plt.subplots(figsize=(5.4, 4.6))
ax.plot(fpr, tpr, "-", color="#2E86C1", lw=2.2, label=f"MLP (AUC = {RESULTS['test_metrics']['auc']:.3f})")
ax.plot(fpr2, tpr2, "-.", color="#E67E22", lw=1.8, label=f"Logistic regression (AUC = {RESULTS['baseline_logreg']['auc']:.3f})")
ax.plot([0, 1], [0, 1], color="0.5", ls=":", label="Random classifier")
ax.set_xlabel("False positive rate"); ax.set_ylabel("True positive rate"); ax.set_title("ROC Curve (Test Set)")
ax.legend(loc="lower right")
save("fig13_roc_curve.png")
# Fig 14: predicted probability distribution
fig, ax = plt.subplots(figsize=(6.4, 4))
ax.hist(p_te[y_te == 0], bins=25, alpha=.9, color=C_NO, edgecolor="black", label="Actually stayed")
ax.hist(p_te[y_te == 1], bins=25, alpha=.75, color=C_YES, edgecolor="black", label="Actually churned")
ax.axvline(0.5, color="black", ls="--", lw=1.5, label="Decision threshold = 0.5")
ax.set_xlabel("Predicted churn probability"); ax.set_ylabel("Number of customers")
ax.set_title("Distribution of Predicted Probabilities (Test Set)"); ax.legend()
save("fig14_probability_distribution.png")

# Permutation feature importance (drop in test AUC when a feature is shuffled)
rng = np.random.default_rng(SEED); base = roc_auc_score(y_te, p_te); imp = {}
for j, name in enumerate(X.columns):
```

```
    drops = []
    for _ in range(5):
        Xp = Xte.copy(); Xp[:, j] = rng.permutation(Xp[:, j])
        drops.append(base - roc_auc_score(y_te, model.predict(Xp,
            verbose=0).ravel()))
    imp[name] = float(np.mean(drops))
top = sorted(imp.items(), key=lambda t: -t[1])[:10]
RESULTS["perm_importance_top10"] = [(k_, round(v, 4)) for k_, v in
        top]
fig, ax = plt.subplots(figsize=(7, 4.4))
ax.barh([t[0] for t in top][::-1], [t[1] for t in top][::-1], color="#4C78A8",
        edgecolor="black")
ax.set_xlabel("Decrease in test AUC after permuting feature");
        ax.set_title("Top 10 Features by Permutation Importance")
ax.grid(axis="y", visible=False)
save("fig15_permutation_importance.png")

with open("results.json", "w") as f:
    json.dump(RESULTS, f, indent=2, default=str)
print("Done. Figures:", sorted(os.listdir(FIG_DIR))).
```

GITHUBLINK :https://github.com/Mithuna713/churn-eda-visualization.